Table of Contents

LLM Basics 2
Chapter 01: LLM Basics **2**
- What is AI Testing? 2
- AI Engineer vs ML Engineer vs DeepSeek Engineer 2
- Popular LLM Examples 3
- LLM vs Chatbot vs Coding Agent 3
- Open vs Closed LLMs 3
- What is an LLM? 3
- How Does Attention Work? 4
- Why Prompt Phrasing Matters 4
- Temperature and Sampling 4
- Interview Questions & Answers 5

Prompt Engineering 6
Chapter 02: Prompt Engineering for QA **6**
- What is Prompt Engineering? 6
- Importance of Prompt Engineering 6
- Types of Prompts 7
- Prompt Strategies with Examples 7
- Modern Prompting Frameworks 8
- Trending Concepts (2025) 8
- Anti-Hallucination Rules 9
- The RICE-POT Framework 9
- 6 Reusable Templates 10
- Project 1: Test Case Generation 10
- Project 2: Selenium Framework 10
- Interview Questions & Answers 11

B.L.A.S.T. Jira Agent 12
Chapter 03: B.L.A.S.T. Jira Test Plan Agent **12**
- What is B.L.A.S.T.? 12
- A.N.T. 3-Layer Architecture 12
- The Data-First Rule 13
- What is a Skill File? 13
- Prompt vs Skill vs AI Agent 13
- Interview Questions & Answers 14

n8n AI Agents 15
Chapter 04: n8n and Local AI Agents **15**
- What is n8n? 15
- Key Workflows 15
- ContentForge Dashboard 15
- Resume Tailor Skill 16
- Interview Questions & Answers 16

LangFlow Agents 17
Chapter 05: AI Agents with LangFlow **17**
- LangFlow vs LangGraph vs LangSmith 17
- Trending AI Agent Concepts (2025) 17
- Flaky Test Analyzer 18
- API Contract Validator 18
- Interview Questions & Answers 18

Content Creation 19

Chapter 06: AI Social Media Content Creation **19**
- Plan Once, Repurpose Everywhere 19
- Platform Templates 19
- Interview Questions & Answers 20

RAG 21

Chapter 07: RAG — Complete Guide **21**
- What is RAG? 21
- Key RAG Terms 21
- Types of RAG 22
- Basic RAG Pipeline 22
- RAG in n8n (No-Code) 23
- Advanced RAG: Hybrid + Rerank 23
- Trending RAG Concepts (2025) 24
- Interview Questions & Answers 24

QABuddy.ai 25

Chapter 08: QABuddy.ai — Multi-Source Hybrid RAG **25**
- The 10 Knowledge Sources 25
- Architecture Decisions 26
- Per-Source Chunking Strategy 26
- Interview Questions & Answers 26

MCP Servers 27

Chapter 09: Build an MCP Server (FastMCP) **27**
- What is MCP? 27
- Three MCP Primitives 27
- Trending MCP Concepts (2025) 28
- Server Requirements 28
- Interview Questions & Answers 28

Python for Testers 29

Chapter 10: Python for Testers **29**
- Python Basics 29
- Keywords, Identifiers, Variables 29
- Literals and Data Types 30
- Interview Questions & Answers 30

Senior Interview Q&A 31

Chapter 11: AI Testing Interview Q&A — Senior Level **31**

E2E Pipeline 33

E2E AI QA Pipeline **33**
- The 8-Step Flow 33
- Interview Questions & Answers 34

Chapter 01: LLM Basics

What is AI Testing?

AI Testing means using Artificial Intelligence tools to help with software testing. Instead of writing every test case by hand, you use smart tools like ChatGPT, Claude, or DeepSeek to create test plans, write automation code, find bugs, and analyze test results faster.

> **Simple Analogy**
> Traditional testing is like writing an essay by hand. AI Testing is like using a smart spell-checker that also suggests better sentences, fixes grammar, and checks facts — all at once.

AI Engineer vs ML Engineer vs DeepSeek Engineer

Role	What They Do	Simple Example
AI Engineer	Builds systems that use AI models. Connects apps to AI APIs. Focuses on product features.	Builds a chatbot for customer support using GPT-4.
ML Engineer	Trains and deploys machine learning models. Works with data, algorithms, and model tuning.	Trains a model to predict house prices from data.
DeepSeek Engineer	Specializes in using DeepSeek models (cost-effective LLM). Optimizes prompts and integrations.	Sets up DeepSeek V3 to generate test cases at 1/10th the cost of GPT-4.

Popular LLM Examples

- GPT-4 / GPT-4o (OpenAI) — General purpose, very smart, widely used.
- Claude 3.5 Sonnet (Anthropic) — Great at reasoning and long documents.
- Gemini 1.5 Pro (Google) — Huge context window, good for code.
- DeepSeek V3 / R1 (DeepSeek) — Very cheap, open-weight, great for coding.
- Llama 3 (Meta) — Open source, can run on your own computer.
- Mistral Large (Mistral AI) — Strong European model.
- Groq (openai/gpt-oss-120b) — Extremely fast inference.

LLM vs Chatbot vs Coding Agent

Term	What It Is	Example
LLM	The brain. A large model that understands and generates text. No memory or tools by itself.	GPT-4, Claude, DeepSeek — just the model.
Chatbot	An app wrapping an LLM with conversation interface and memory.	ChatGPT, Claude.ai — you chat with them.
Coding Agent	An AI that writes, runs, and fixes code using tools.	GitHub Copilot, Cursor, LangFlow agents.

Open vs Closed LLMs

Type	Meaning	Examples	Pros	Cons
Open-Source	Model weights	Llama 3, Mistral,	Free, private,	Needs powerful

	public. Download and run locally.	DeepSeek V3	customizable	hardware
Closed	Owned by company. Access via API only.	GPT-4, Claude 3.5, Gemini	Easy to use, always updated	Costs money, less control

What is an LLM?

An LLM is like a super-smart text predictor. It reads your words and guesses the next word, one at a time. It learned patterns from reading billions of sentences during training.

How Does Attention Work?

- Self-Attention: Every word looks at every other word and asks 'How related are we?'
- Multi-Head Attention: Multiple attention layers run together to catch different relationships.
- Positional Encoding: Words get a 'position number' so the model knows order.

Why Prompt Phrasing Matters

Because the model weighs every word against every other word, vague words like 'be thorough' add noise without direction.

> **Quick Tip**
> Use clear instructions. Instead of 'be thorough', say 'cover boundary values, negative inputs, and security cases.'

Temperature and Sampling

- Temperature = 0: Same answer every time. Best for test cases.
- Temperature = 0.7: More creative. Good for brainstorming.
- Temperature > 1: Very random. Avoid for testing tasks.

```
temperature=0   # Use this for deterministic test case generation
```

Interview Questions & Answers

Q: What is AI Testing in simple words?
A: Using smart tools like ChatGPT to create test cases, write automation, and find bugs faster.

Q: What is the difference between an AI Engineer and an ML Engineer?
A: AI Engineer builds products using ready-made models. ML Engineer trains new models from data.

Q: What is a DeepSeek Engineer?
A: A specialist who uses DeepSeek models to build cheap, powerful AI solutions for coding and testing.

Q: What is the difference between an LLM and a Chatbot?
A: LLM is just the brain. Chatbot is brain + memory + conversation interface.

Q: What is a Coding Agent?
A: An AI that writes, runs, and fixes code using tools like file editors and terminals.

Q: Give examples of Open-Source and Closed LLMs.
A: Open: Llama 3, Mistral, DeepSeek V3. Closed: GPT-4, Claude 3.5, Gemini.

Q: What is self-attention?
A: Every word in a sentence looks at every other word to understand relationships.

Q: Why should QA engineers care about temperature?
A: Temperature controls consistency. For test cases, we want the same output every time (temp=0).

Q: What does 'attention is all you need' mean?
A: It is the title of the famous paper that introduced Transformers. Attention alone is powerful enough.

Q: How do LLMs differ from search engines?
A: Search engines find existing pages. LLMs generate NEW text by predicting words.

Chapter 02: Prompt Engineering for QA

■ What is Prompt Engineering?

Prompt Engineering is the skill of writing clear instructions to an AI so it gives you the best possible answer. It is like giving directions to a taxi driver. The more specific you are, the faster you reach your destination.

■ Importance of Prompt Engineering

- Better Quality: A good prompt gives 95% useful output. A bad prompt gives 40%.
- Saves Time: Less back-and-forth correcting the AI.
- Reduces Hallucination: Clear rules stop the AI from making things up.
- Consistency: Same prompt = same quality every time.
- Cost Savings: Fewer tokens wasted on wrong answers.

■ Types of Prompts

Type	Description	Example
Instruction	Direct command telling the AI what to do.	'Generate 10 test cases for login.'
Question	Asking the AI a specific question.	'What are boundary values for password?'
Context	Giving background info before the task.	'Here is the PRD. Generate API tests.'
Role	Telling the AI to act as someone.	'Act as a senior QA engineer...'
Template	Fill-in-the-blanks format.	'Given [FEATURE], create [TYPE] tests.'
Constraint	Adding rules and limits.	'Do not include UI tests. Only API.'

■ Prompt Strategies with Examples

1. Zero-Shot Prompting

Give the AI a task with NO examples. It relies only on training.

```
Write test cases for a user registration form.
```

2. Few-Shot Prompting

Give 1-3 examples before the task. The AI copies the pattern.

```
Example 1: Login form -> TC001 Valid login (Positive)\nExample 2: Empty password -> TC002
Error message (Negative)\nNow write test cases for a signup form.
```

3. Chain-of-Thought (CoT)

Ask the AI to think step-by-step before giving the final answer.

```
A cart has items costing Rs.100, Rs.200, Rs.300.\nA 10% discount applies if total >
Rs.500.\nThink step by step. What is the final amount?
```

4. Tree-of-Thought (ToT)

The AI explores multiple solutions like branches of a tree, then picks the best.

```
Test a payment gateway. Consider 3 approaches: happy path, failure path, edge cases.\nList
top 3 risks per approach. Pick the highest risk.
```

5. Role-Based Prompting

Assign a persona so the AI answers from that perspective.

```
You are a senior QA automation engineer with 10 years experience.\nReview this API design and list 5 security risks.
```

Modern Prompting Frameworks

Framework	What It Does	Best For
RICE-POT	7-part prompt: Role, Instructions, Context, Example, Parameters, Output, Tone.	Complex QA tasks, test case generation.
RTCFR	Lightweight 5-part: Role, Task, Constraints, Format, Requirements.	Quick daily repeatable tasks.
RISE-CEPT	Role, Instructions, Spec, Example, Constraints, Output, Process, Tone.	Technical tool building, coding.
APE	Action, Purpose, Expectation. Simple 3-part.	Quick instructions, simple queries.
COSTAR	Context, Objective, Style, Tone, Audience, Response format.	Marketing, multi-audience tasks.

Trending Concepts in Prompt Engineering (2025)

- Structured Output / JSON Mode: Force the AI to reply in valid JSON. Useful for APIs and databases.
- Function Calling: The AI decides which function to call and with what arguments. Like a smart router.
- Prompt Chaining: Breaking a big task into smaller prompts where output of one becomes input of the next.
- ReAct Pattern: Reason + Act. The AI thinks, then uses a tool, then thinks again. Great for agents.
- Reflection Pattern: The AI reviews its own answer and improves it before showing you.
- System Prompts vs User Prompts: System = rules and persona. User = the actual question.

Anti-Hallucination Rules

- Only use information you give it (PRD, API docs, screenshots).
- Never invent features, error codes, or UI elements.
- Never assume 'typical' behavior.
- If info is missing, say exactly: 'Insufficient information to determine.'
- Label guesses as: 'Inference (low confidence)'

> **Why This Matters**
> Without these rules, the AI might create a test case for a 'Delete User' button that does not exist.

The RICE-POT Framework

Letter	Component	Simple Meaning
R	Role	Who should the AI act as?
I	Instructions	What steps should it follow?
C	Context	What background info does it need?
E	Example	Show one sample row or code snippet.

P	Parameters	Quality rules and anti-hallucination blocks.
O	Output	Exactly what format? CSV? Markdown? Code only?
T	Tone	Technical? Friendly? Output-only?

> **Most Skipped Letter = P**
> Without Parameters, the AI invents fields and IDs. Output looks correct but will fail in real testing.

6 Reusable Templates

1. Basic Test Case Generation
2. PRD to Test Cases — functional, negative, boundary, edge.
3. API Test Generation
4. Negative-Only Suite
5. Security Tests — OWASP Top 10 aligned.
6. Regression Suite

Project 1: Test Case Generation

- Input: PDF PRD or API doc -> Output: CSV with exact columns.
- Why CSV? Imports into Jira, TestRail, qTest, Zephyr.
- Trust Check: Every row must have a 'Traceability' column.

Project 2: Selenium Framework from Prompt

- Stack: Maven + Java 11 + Selenium 4.25 + TestNG 7.10
- Pattern: PageFactory POM + explicit waits + XPath-only
- Config: config.properties with REPLACE_WITH_... placeholders
- Default: Headless Chrome for CI stability

Interview Questions & Answers

Q: What is Prompt Engineering?
A: The art of writing clear instructions to AI for the best answer.

Q: Why is prompt engineering important for QA?
A: A good prompt gives ready-to-use test cases. A bad prompt gives garbage.

Q: What is Zero-Shot prompting?
A: Giving the AI a task with zero examples. Uses only training knowledge.

Q: What is Few-Shot prompting?
A: Giving 1-3 examples so the AI copies the pattern.

Q: What is Chain-of-Thought prompting?
A: Asking the AI to think step-by-step before answering. Good for logic tasks.

Q: What is Tree-of-Thought prompting?
A: The AI explores multiple solution paths and picks the best one.

Q: What is Role-Based prompting?
A: Telling the AI to act as an expert to improve quality and tone.

Q: What is the RICE-POT framework?
A: A 7-part prompt template: Role, Instructions, Context, Example, Parameters, Output, Tone.

Q: What is the difference between RICE-POT and RTCFR?
A: RICE-POT is full 7-letter for complex tasks. RTCFR is lighter 5-part for quick tasks.

Q: What are anti-hallucination rules?
A: Rules forcing AI to only use provided info. Cannot invent features or error codes.

Q: What happens if you skip the 'P' in RICE-POT?
A: The AI starts inventing things. Test cases look real but reference non-existent buttons.

Q: Name three modern prompting frameworks.
A: RICE-POT, RTCFR, RISE-CEPT, APE, COSTAR. (Any three)

Q: What is JSON Mode in prompting?
A: Forcing the AI to output valid JSON instead of free text. Essential for APIs.

Q: What is Function Calling?
A: The AI decides which function to call and what arguments to pass. Like a smart router.

Q: What is the ReAct pattern?
A: Reason + Act. The AI thinks, uses a tool, thinks again. Common in AI agents.

Chapter 03: B.L.A.S.T. Jira Test Plan Agent

■ What is B.L.A.S.T.?

Phase	What You Do	Simple Analogy
B — Blueprint	Ask 5 discovery questions. Define data schema FIRST.	Architect draws house plan before building.
L — Link	Test all API keys and connections before writing logic.	Check water and electricity before moving in.
A — Architect	Build in 3 layers: SOPs -> Navigation -> Tools.	Foundation -> Walls -> Furniture.
S — Stylize	Make outputs pretty. Get feedback.	Paint walls and add decorations.
T — Trigger	Deploy to cloud. Set up cron jobs or webhooks.	Hand over keys and set up auto lights.

■ A.N.T. 3-Layer Architecture

Layer	Name	What It Does	Analogy
Layer 1	Architecture	Written SOPs in Markdown. Defines goals, inputs, edge cases.	The recipe book.
Layer 2	Navigation	Decides which recipe to use and which cook to call.	The head chef.
Layer 3	Tools	Deterministic Python scripts that do one job perfectly.	The cooks.

> **Self-Annealing (Repair Loop)**
> When something breaks: 1) Read the error. 2) Fix the script. 3) Test. 4) Update the SOP so it never repeats.

■ The Data-First Rule

Before writing code, define input and output shapes. This goes into gemini.md. It is the LAW of your project.

■ What is a Skill File?

A Skill File is a reusable instruction document that teaches an AI how to do a specific job. Like a training manual for a new employee.

- Reusable: Use the same skill for multiple projects.
- Consistent: Same input always produces same quality output.
- Shareable: Team members can use your skill without re-explaining.
- Version-controlled: Track improvements over time.

> **Why Skill Files Matter**
> Instead of typing the same long prompt daily, save it as a SKILL.md. The AI reads it once and remembers the job.

■ Prompt vs Skill vs AI Agent

Term	What It Is	Lifespan	Example
Prompt	A single instruction typed into an AI chat.	One conversation	'Write test cases for login.'
Skill	A saved, reusable set of instructions.	Permanent file	SKILL.md for generating Selenium frameworks.
AI Agent	A system that reads skills, makes decisions, and runs workflows automatically.	Running app	n8n workflow that generates test plans without humans.

■ Interview Questions & Answers

Q: What does B.L.A.S.T. stand for?
A: Blueprint, Link, Architect, Stylize, Trigger. A 5-phase method for reliable AI automation.

Q: What is the A.N.T. architecture?
A: Layer 1 = SOPs, Layer 2 = Navigation, Layer 3 = Tools.

Q: What is the Data-First Rule?
A: Define input/output data shapes BEFORE writing code. Schema goes in gemini.md.

Q: What is a Skill File?
A: A reusable instruction document teaching an AI to do a task consistently.

Q: What is the difference between a Prompt and a Skill?
A: A prompt is one-time. A skill is saved and reusable anytime.

Q: What is an AI Agent?
A: A system reading skills, making decisions, using tools, and running workflows automatically.

Q: What is Self-Annealing?
A: Fix a bug, test it, then update the architecture SOP so the error never repeats.

Q: Why is the 'Link' phase important?
A: If API keys are broken, everything built after fails. Test links FIRST.

Q: What is gemini.md?
A: The project constitution. Holds data schemas, rules, and architectural decisions.

Q: How does B.L.A.S.T. help QA engineers?
A: Provides a structured, repeatable way to build AI tools for test plan generation.

Chapter 04: n8n and Local AI Agents

■ What is n8n?

n8n is a visual workflow builder. You drag and drop blocks to connect apps like Jira, Google Sheets, Slack, and AI models. No coding required.

■ Key Workflows

- QA Buddy — Chat-triggered QA assistant using Groq.
- Jira Agent — Chat agent that creates Jira tickets.
- PRD to Test Cases — Upload PRD, get test cases in Google Sheets.
- Social Media Agent — Scheduled daily posts to Sheets + Drive.

■ ContentForge Dashboard

- Groq writes blog posts, LinkedIn posts, and scripts.
- Gemini generates images.
- Everything saves locally: Excel calendar + image files.
- API keys stay in .env.local — never commit them.

■ Resume Tailor Skill

A 4-step honest resume builder:

1. Score your resume out of 10.
2. Find keyword gaps vs job description (ATS check).
3. Confirmation gate: Only add confirmed skills. No lying.
4. Build a clean .docx that passes ATS scanners.

■ Interview Questions & Answers

Q: What is n8n?
A: A visual, no-code workflow builder connecting apps and AI with drag-and-drop.

Q: Name three n8n workflows.
A: QA Buddy, Jira Agent, PRD to Test Cases, Social Media Agent. (Any three)

Q: What is ContentForge?
A: A local Next.js dashboard using Groq for writing and Gemini for images.

Q: What is the Resume Tailor Skill?
A: A 4-phase workflow scoring resumes, checking ATS gaps, confirming skills honestly, building .docx.

Q: Why is the confirmation gate important?
A: Prevents AI from adding fake skills. Only real, confirmed skills are added.

Q: What is a Schedule Trigger in n8n?
A: A timer running workflows at set times, like a cron job.

Q: What is the benefit of local-first tools?
A: Data never leaves your computer. No cloud bills, no privacy risks.

Q: How does the Social Media Agent work?
A: Wakes on schedule, drafts posts via LLM, structures output, writes to Sheets and Drive.

Q: What is an ATS gap analysis?
A: Comparing resume keywords against a job description to find missing matches.

Q: Why use three model nodes in one workflow?
A: Swappable backups. If one fails or rate-limits, another takes over.

Chapter 05: AI Agents with LangFlow

LangFlow vs LangGraph vs LangSmith

Tool	What It Does	Best For	Code Needed?
LangFlow	Visual drag-and-drop builder. Publish as REST API.	Fast prototyping, non-coders.	Little to none
LangGraph	Code-first agent graphs with loops, memory, human approval.	Production systems.	Yes — Python/JS
LangSmith	Tracing and evaluation dashboard.	Debugging, testing, cost tracking.	No — just hook in

> **They Stack, Not Compete**
> LangGraph builds the brain. LangFlow lets non-coders adjust it visually. LangSmith watches everything and finds bugs.

Trending AI Agent Concepts (2025)

- Agentic AI: AI systems that plan, decide, and act with minimal human help.
- Multi-Agent Systems: Multiple AI agents working together, each with a specialty.
- ReAct Agents: Reason + Act loop. Think, use tool, observe result, think again.
- Tool Use: Agents calling APIs, running code, querying databases as part of their work.
- Memory Types: Short-term (chat history) and Long-term (vector DB storing past learnings).
- Agent Frameworks: CrewAI (team of agents), AutoGen (Microsoft), LangGraph (stateful graphs).
- Human-in-the-Loop: Agent pauses for human approval on critical decisions.

Flaky Test Analyzer

- Flaky Test = Passes in one build, fails in another. Recommend rerun.
- Consistent Failure = Fails in both builds. Send to engineering.
- No human comparison needed. The agent decides.

API Contract Validator

- Give it a GET URL + JSON Schema.
- It calls the endpoint and compares the real response.
- Output: Human-readable diff like 'status is now a number, was a string.'

Interview Questions & Answers

Q: What is LangFlow?
A: A visual, low-code tool to build LLM apps by dragging nodes. Every flow becomes a REST API.

Q: What is the difference between LangFlow and LangGraph?
A: LangFlow is visual and no-code. LangGraph is code-first with full control.

Q: What is LangSmith used for?
A: Observability — traces every AI run, shows token costs, latency, and helps debug failures.

Q: Why would a QA engineer use LangSmith?
A: To build evaluation datasets, catch regressions, and track agent behavior.

Q: What is Agentic AI?
A: AI systems that plan, decide, and act on their own with minimal human supervision.

Q: What is a Multi-Agent System?
A: Multiple AI agents working together, each handling a different specialty task.

Q: What is the ReAct pattern?
A: Reason + Act. The AI thinks, uses a tool, observes the result, and thinks again.

Q: What is Human-in-the-Loop?
A: The agent pauses and asks for human approval before making critical decisions.

Q: What is a Flaky Test?
A: A test that sometimes passes and sometimes fails without code changes.

Q: What is API Contract Validation?
A: Checking if a live API response still matches its agreed JSON schema.

Q: Why use an LLM for contract validation?
A: It gives a human-readable diff and explanation, not just pass/fail.

Q: What does 'coverage is not confidence' mean?
A: Traditional coverage counts lines. For AI apps, you need evaluation datasets in LangSmith to truly trust the system.

Chapter 06: AI Social Media Content Creation

The Golden Rule: Plan Once, Repurpose Everywhere

Do not write 7 different posts for one idea. Write ONE plan, then reshape it for every platform.

```
IDEA: _____ (one sentence only)\n\nHOOK   (stop scrolling in 3
seconds): _____\nSTORY  (Problem -> Tension -> Solution -> Proof):
_____\nOFFER  (exactly ONE thing to do): _____\n\nSCREENSHOT LINE
(quotable truth): _____\nHONEST CAVEAT (what could go wrong):
_____
```

Platform Templates

1. YouTube Video — 8-beat script structure.
2. Instagram Reel — Hook in first 3 seconds, 15-30s total.
3. Instagram Post — Single image + strong caption.
4. Instagram Carousel — Swipeable slides with clear CTA.
5. Medium Article — Long-form, SEO-friendly.
6. Blog Post — Tutorial style with code examples.
7. LinkedIn Post — Professional story + clear takeaway.

Interview Questions & Answers

Q: What does 'Plan once, repurpose everywhere' mean?
A: Write one Hook-Story-Offer plan, then adapt it into 7 formats.

Q: What are the 3 parts of the planning skeleton?
A: Hook, Story, Offer.

Q: Why should the hook never be a statistic?
A: Statistics do not stop scrolling. A personal story or bold statement does.

Q: What is an Honest Caveat?
A: Mentioning what could go wrong. This builds trust with the audience.

Q: How many platform templates are provided?
A: Seven: YouTube, IG Reel, IG Post, IG Carousel, Medium, Blog, LinkedIn.

Q: What is the 8-beat video script structure?
A: Hook -> Promise -> Why Now -> Definition -> How-To -> Payoff -> Lesson -> CTA.

Q: Why is consistency important?
A: It makes your channel recognizable. Viewers know what to expect.

Q: How do you use templates with an AI assistant?
A: Fill the plan, paste with the platform template, and say 'write using this plan and these rules.'

Q: What makes a good Screenshot Line?
A: A single quotable sentence people can screenshot and share.

Q: What is the brand voice summary?
A: Talk like a smart friend figuring it out in real time — simple ideas, honest pros/cons, numbered steps.

Chapter 07: RAG — Complete Guide

■ What is RAG?

RAG stands for Retrieval-Augmented Generation. It is like giving the AI an open-book exam. Instead of trusting its memory, you let it search YOUR documents first, then answer using only what it found.

> **Why RAG Matters**
> LLMs hallucinate and have a knowledge cutoff. RAG grounds answers in YOUR up-to-date documents.

■ Key RAG Terms

Term	Simple Definition
Embedding	Turning text into a list of numbers that capture its meaning.
Vector Store	A special database that stores embeddings and finds similar ones using math.
Chunking	Breaking large documents into smaller pieces so the AI can search them.
Retrieval	Finding the most relevant chunks for a given question.
Generation	The AI writing an answer based on the retrieved chunks.
Grounding	Connecting the AI's answer to real source documents.
Hallucination	When the AI makes up facts that are not in the source documents.
Top-K	The number of best-matching chunks to retrieve (e.g., top 4).
Cosine Similarity	A math score from 0 to 1 showing how similar two text pieces are.
Context Window	The maximum amount of text the AI can read at one time.

■ Types of RAG

Type	What It Does	When to Use
Naive RAG	Basic: chunk -> embed -> store -> retrieve -> generate.	Small datasets, simple questions, getting started.
Advanced RAG	Adds query rewriting, hybrid search, reranking, and better chunking.	Large datasets, production systems, accuracy matters.
Modular RAG	Mix and match components: different chunkers, retrievers, and generators.	When you need flexibility to swap parts.
Graph RAG	Builds a knowledge graph from documents. Answers complex relational questions.	Documents with many connections (e.g., org charts, systems).
Corrective RAG (CRAG)	Checks if retrieved chunks are good. Falls back to web search if they are not.	When documents might not have the answer.
Self-RAG	The AI evaluates its own retrieval	When quality control is critical.

	and decides whether to retrieve more or answer.	
Fusion RAG	Combines multiple retrieval methods (keyword + semantic + graph) into one answer.	When one search method is not enough.

■ Basic RAG Pipeline

```
PDF -> Chunk (small pieces)\n -> Embed (turn text into numbers)\n -> Store in ChromaDB (vector database)\n -> Ask a question\n -> Retrieve top matches\n -> Groq generates answer from those matches only
```

- Embedding Model: nomic-embed-text via local Ollama (free, offline).
- Vector Store: ChromaDB with cosine similarity.
- LLM: Groq openai/gpt-oss-120b.
- UI: React + Express shows every step.

■ RAG in n8n (No-Code)

Phase 1: Upload PDF -> Split -> Embed -> Save to Pinecone.

Phase 2: Ask question -> AI searches Pinecone -> Answers using found chunks.

■ Advanced RAG: Hybrid + Rerank

```
CSV -> chunk -> bge-m3 (dense + sparse vectors) -> Qdrant\nQuery -> rewrite x3 -> dense + sparse search -> RRF fuse (k=60)\n -> bge-reranker-v2-m3 (top-12 -> top-6) -> Groq -> cited answer
```

Technique	Simple Explanation
Hybrid Search	Dense finds meaning. Sparse finds exact words. Both together = best results.
RRF Fusion	Merges two search results fairly without complex math.
Cross-Encoder Reranker	Re-reads top results with full question context. Biggest quality boost.
Query Rewriting	AI rephrases your question 3 ways to catch more matches.

> **Biggest Lever**
> Chunking strategy matters more than the AI model. Better chunks = better answers.

■ Trending RAG Concepts (2025)

- Contextual Chunking: Adding section titles and summaries to each chunk for better retrieval.
- Late Chunking: Embedding full documents first, then splitting. Keeps more context per chunk.
- Hypothetical Document Embeddings (HyDE): The AI generates a fake perfect answer, then searches for documents matching it.
- Re-ranking with LLM: Using a large model to score retrieved chunks instead of a small cross-encoder.
- Multi-modal RAG: Retrieving from images, videos, and audio — not just text.
- Agentic RAG: An agent decides WHEN to search, WHAT to search, and HOW MANY times to search.

Interview Questions & Answers

Q: What is RAG in simple words?
A: The AI searches your documents first, then answers using only what it found. Like an open-book exam.

Q: Why use RAG instead of just asking the LLM?
A: LLMs hallucinate and have knowledge cutoffs. RAG grounds answers in YOUR documents.

Q: What is an embedding?
A: A list of numbers representing the meaning of text. Similar texts have similar numbers.

Q: What is a vector database?
A: A database storing embeddings and finding closest matches using math.

Q: What is the difference between Naive and Advanced RAG?
A: Naive uses basic search. Advanced adds hybrid search, reranking, and query rewriting.

Q: What is hybrid search?
A: Combining dense (semantic) and sparse (keyword) search for better results.

Q: What does a reranker do?
A: Re-reads top results with the full question and re-scores them for accuracy.

Q: What is query rewriting?
A: The AI creates 3 versions of your question to improve search recall.

Q: What is Graph RAG?
A: Building a knowledge graph from documents to answer complex relational questions.

Q: What is Self-RAG?
A: The AI evaluates its own retrieval quality and decides whether to search more or answer.

Q: What is Corrective RAG (CRAG)?
A: Checks if retrieved chunks are relevant. Falls back to web search if they are not.

Q: What is cosine similarity?
A: A math score from 0 to 1 measuring how similar two embeddings are. 1 = identical meaning.

Q: What is HyDE?
A: Hypothetical Document Embeddings. The AI generates a fake perfect answer, then searches for real documents matching it.

Q: Why is chunking the biggest lever in RAG?
A: Right chunk size = right context. Too big = noise. Too small = missing context.

Q: What is Agentic RAG?
A: An AI agent decides when to search, what to search, and how many times — instead of a fixed pipeline.

Chapter 08: QABuddy.ai — Multi-Source Hybrid RAG

What is QABuddy.ai?

A self-hosted QA knowledge brain. Ask one question. Get one cited answer backed by 10 real data sources.

The 10 Knowledge Sources

1. Selenium framework code (Java)
2. Playwright framework code (TypeScript)
3. ~5,000 test cases (CSV/XLSX)
4. JIRA tickets (JSON dumps)
5. Company documents (PDF/Markdown)
6. Figma designs (Phase 2)
7. Meeting transcripts
8. Lucid chart exports
9. PRD / SRS / BRD / FRD documents
10. Jenkins failure logs

Architecture Decisions

- Embedding: BAAI/bge-m3 — dense + sparse vectors in one pass.
- Vector DB: Qdrant (Docker) — hybrid search, low RAM, snapshots.
- Reranker: bge-reranker-v2-m3 — top-12 reranked to best 6.
- Generator: Groq openai/gpt-oss-120b.
- Modes: Answer / Generate Test Cases / Review / RCA.

Per-Source Chunking Strategy

Source	Strategy	Size
Code repos	Structure-aware (method/class per chunk)	300-500 tokens
Test cases	1 row = 1 chunk	Row size
JIRA tickets	1 ticket = 1 chunk	Up to 1,200 tokens
Docs / PRD	Heading-aware recursive split	512 tokens / 76 overlap
Transcripts	Speaker-turn windows	800-1,000 tokens
Jenkins logs	Failure block + summary per build	300-600 tokens

> **Trust First**
> Every answer includes citations. If confidence is low, it says 'Not found in KB' — never guesses.

Interview Questions & Answers

Q: What is QABuddy.ai?
A: A self-hosted QA knowledge brain answering questions using 10 real data sources with citations.

Q: Name 5 of the 10 knowledge sources.
A: Selenium repo, Playwright repo, test cases, JIRA tickets, company docs, transcripts, Jenkins logs, PRDs. (Any 5)

Q: Why use bge-m3 for embeddings?
A: Outputs dense + sparse vectors in one pass. No need for two separate models.

Q: Why is Qdrant chosen over ChromaDB?
A: True hybrid search, lower RAM, supports server mode for 24/7 deployment.

Q: What is RRF (Reciprocal Rank Fusion)?
A: Merges two ranked search results into one fair list.

Q: What are the 4 modes of QABuddy?
A: Answer, Generate Test Cases, Review, and RCA.

Q: Why is per-source chunking important?
A: Code needs small atomic chunks. Documents need overlap. One size does not fit all.

Q: What makes QABuddy 'trust first'?
A: Every chunk has breadcrumb metadata. Below-threshold answers say 'not found' instead of guessing.

Q: What is the estimated server cost?
A: 8GB / 4 vCPU DigitalOcean droplet ~ $48-56 per month.

Q: How does QABuddy handle private data?
A: Data folders are gitignored. Only READMEs and fixtures are committed. Company data never hits GitHub.

Chapter 09: Build an MCP Server (FastMCP)

What is MCP?

MCP = Model Context Protocol. It is a USB port for AI. It lets ChatGPT, Claude, or any AI talk to YOUR local files and tools safely.

Three MCP Primitives

Primitive	Who Uses It	Simple Example
Tools	The AI (LLM)	Search test cases by keyword.
Resources	The App / URI	Get all test cases for the 'Login' module.
Prompts	The User	'Review this test case for me' template.

Trending MCP Concepts (2025)

- stdio Transport: Default method. Server communicates over input/output pipes. Simple and fast.
- SSE Transport: Server-Sent Events over HTTP. Good for web-based integrations.
- Authentication: MCP supports OAuth and API key authentication for secure connections.
- Sampling: The server can request the AI model to generate text on its behalf.
- Roots: A way for the client to tell the server which directories or URIs it can access.
- Tool Schema: FastMCP auto-generates JSON schemas from Python type hints. The AI knows exactly what each tool needs.

Server Requirements

- Load CSV once at startup. Reuse across calls.
- At least 3 Tools: search, get by ID, stats.
- At least 3 Resources: schema, all data, module filter.
- At least 2 Prompts: review test case, generate regression suite.
- Type hints + docstrings required. FastMCP reads them.
- Log to stderr only. stdout breaks the connection.
- Handle errors gracefully. Never crash.

> **FastMCP Tip**
> Keep under 250 lines. Test with MCP Inspector before connecting to Claude Desktop.

Interview Questions & Answers

Q: What is MCP?
A: Model Context Protocol. A standard way for AI models to connect to your local data and tools.

Q: What are the three MCP primitives?
A: Tools (model-invoked), Resources (app-fetched by URI), Prompts (user-invoked).

Q: Who invokes a Tool?
A: The AI model decides when to call a tool based on the user's question.

Q: Who invokes a Resource?
A: The application fetches it by URI. The model does not decide this.

Q: Who invokes a Prompt?
A: The user explicitly requests it, like clicking a template.

Q: Why log to stderr only in MCP?
A: stdout is used for JSON-RPC communication. Printing to stdout corrupts the connection.

Q: What is FastMCP?
A: A Python library making MCP server building easy. Add @mcp.tool decorators and it handles the protocol.

Q: Why are type hints important in FastMCP?
A: FastMCP auto-generates JSON schemas and descriptions from type hints and docstrings.

Q: What is the MCP Inspector?
A: A debugging tool to manually test your MCP server's tools, resources, and prompts.

Q: What is a templated resource URI?
A: A resource with variables like testcases://module/{name}. The app fills {name} at runtime.

Q: What is stdio transport?
A: Server communicates over standard input/output pipes. Default and most common.

Q: What is SSE transport?
A: Server-Sent Events over HTTP. Useful for web-based MCP integrations.

Q: What is Sampling in MCP?
A: The server can ask the AI model to generate text on its behalf.

Q: What are Roots in MCP?
A: The client tells the server which directories or URIs it is allowed to access.

Q: Why keep the MCP server under 250 lines?
A: To keep it simple, readable, and maintainable.

Chapter 10: Python for Testers

Python Basics (Exercises 1-3)

- print() — show text on screen. Can print many things at once.
- Comments — lines starting with # are ignored. Use them to explain code.
- sep= and end= — control how print separates and ends lines.

```
print("Hello", "World", sep="-")   # Hello-World\nprint("Line 1", end=" ")\nprint("Line 2")
# Line 1 Line 2
```

Keywords, Identifiers, Variables

- Keywords — 35 reserved words Python knows (if, else, for, while, class, etc.).
- Identifiers — Names for variables and functions. Start with letter or _, no spaces, no keywords.
- Variables — Boxes holding data. Python guesses the type automatically (dynamic typing).

```
name = "Rahul"      # str (text)\nage = 30          # int (whole number)\nsalary = 50000.50
# float (decimal)\nprint(type(name)) # <class 'str'>
```

Identifier Cheat Sheet
Start with letter or _ · no spaces · no keywords · case-sensitive · use snake_case per PEP 8.

Literals and Data Types

- Literals — Raw values: 42, 3.14, 'hello', True.
- Built-in functions: pow(2, 3) = 8, abs(-5) = 5.
- input() reads keyboard input. Always returns text.
- Use int() to convert text to numbers before math.

```
age = input("Enter age: ")    # Returns "25" (text)\nage = int(age)              #
Converts to 25 (number)\nprint(age + 5)               # 30
```

Interview Questions & Answers

Q: What is the difference between a keyword and an identifier?
A: Keywords are reserved words Python knows. Identifiers are names YOU create.

Q: What is dynamic typing in Python?
A: Python guesses the data type automatically. No need to declare 'this is a string'.

Q: What is snake_case?
A: Naming style: lowercase words separated by underscores. Example: my_variable_name.

Q: What does input() return?
A: Always a string. Use int() or float() to convert before math.

Q: What is the output of type(3.14)?
A: <class 'float'> because 3.14 is a decimal number.

Q: What is a literal?
A: A raw value in code: 42 (int), 'hello' (str), True (bool).

Q: What is the BODMAS rule?
A: Order of math: Brackets, Orders, Division, Multiplication, Addition, Subtraction.

Q: What happens if you do '5' + 3 in Python?
A: TypeError! Cannot add text and number. Convert first: int('5') + 3 = 8.

Q: What are the 7 identifier rules?
A: 1) Letter or _ start. 2) No spaces. 3) No special symbols. 4) Case-sensitive. 5) No keywords. 6) Meaningful. 7) snake_case.

Q: Why should testers learn Python?
A: Easy to read, great testing libraries (Selenium, Playwright, pytest), main language for AI/ML tools.

Chapter 11: AI Testing Interview Q&A — Senior Level

This chapter contains interview questions commonly asked in MNC organizations for software testers with 7-8 years of experience.

■ Interview Questions & Answers

Q: How do you evaluate if an AI-generated test case is correct?
A: I check traceability to source documents, verify boundary values, ensure negative cases exist, and manually validate at least 10% of cases against the actual application.

Q: What metrics do you use to measure AI testing effectiveness?
A: Coverage percentage, defect detection rate, false positive rate of AI suggestions, time saved vs manual creation, and cost per test case generated.

Q: How do you prevent AI hallucination in test automation?
A: Use anti-hallucination rules, temperature=0, structured output formats, traceability columns, and human review gates before execution.

Q: Explain how you would set up an AI agent for regression test selection.
A: Use RAG to index code changes and historical test failures. The agent retrieves related test cases, ranks them by risk, and suggests a minimized regression suite.

Q: What is the biggest risk of using AI in testing?
A: Over-reliance without verification. AI can generate confident-looking but incorrect test cases. Always have a human review gate.

Q: How do you handle data privacy when using cloud LLMs for testing?
A: Use local models (Ollama) for sensitive data, anonymize test data before sending to APIs, ensure no PII in prompts, and review vendor data policies.

Q: Describe a real project where you used AI to improve testing.
A: I used RICE-POT prompts with DeepSeek to generate 200 API test cases from a PRD in 30 minutes. Manual effort would have taken 2 days. I validated 20% manually and found 95% accuracy.

Q: How do you test an AI-powered feature itself?
A: Test for bias, edge cases, unexpected inputs, performance under load, fallback behavior when AI is unavailable, and consistency of outputs across runs.

Q: What is the difference between testing WITH AI and testing AI?
A: Testing WITH AI means using AI tools to help your testing. Testing AI means validating the AI system's behavior, accuracy, fairness, and robustness.

Q: How do you version control AI prompts in a team?
A: Store prompts as SKILL.md files in Git. Version them like code. Use A/B testing to compare prompt versions and track which performs better.

Q: What would you do if the AI keeps generating the same incorrect pattern?
A: Add the incorrect pattern to the 'Don't' list in the prompt. Provide a corrected example. Tighten the Parameters section. If needed, switch to Few-Shot prompting.

Q: How do you integrate AI test generation into a CI/CD pipeline?
A: Use LangFlow API or n8n webhook triggers. On code commit, fetch changed files, run AI test generation, output test cases to the test folder, and trigger the test suite automatically.

Q: What is your approach to evaluating RAG system quality?
A: Use golden question sets with known correct answers. Measure hit rate (did it find the right chunk?), answer relevance, citation accuracy, and latency per query.

Q: How do you handle flaky tests in an AI-driven pipeline?
A: Use the Flaky Test Analyzer agent. Compare baseline vs candidate results. Quarantine flaky tests, investigate environment issues, and re-run with deterministic settings.

Q: What are the cost considerations when scaling AI testing?
A: Use cheaper models (DeepSeek) for generation, expensive models (GPT-4) only for review. Implement caching for repeated queries. Use local embeddings to avoid API costs. Monitor token usage per workflow.

E2E AI QA Pipeline

The 8-Step Flow

1. Step 1: Fetch Jira Stories — JQL query via AI agent.
2. Step 2: Process One by One — Pick story VWO-109.
3. Step 3: Create Test Plan — Use RAG with historical plans.
4. Step 4: Generate Test Cases — Use RAG with previous patterns.
5. Step 5: Convert to Automation — Agent writes Playwright .md flow files.
6. Step 6: Execute via Browser Bash — AI runs .md files automatically.
7. Step 7: Generate result.json — Pass/fail, logs, errors.
8. Step 8: Analyze with AI — Flakiness check -> RCA -> Triage -> Dashboard.

Final Outcome
A reusable, intelligent pipeline: Jira -> Plan -> Cases -> Automation -> Run -> RCA -> Dashboard. Zero manual test-case writing for standard flows.

Interview Questions & Answers

Q: What is the E2E AI QA Pipeline?
A: An 8-step automated workflow from Jira tickets to test plans, cases, automation, execution, and reporting.

Q: What is JQL?
A: Jira Query Language. Like SQL for Jira. Used to search and filter tickets programmatically.

Q: What does Browser Bash do?
A: A CLI tool reading .md automation files and executing them using an AI model to control the browser.

Q: Why use RAG in Steps 3 and 4?
A: RAG pulls historical test plans and cases as examples, making new output more accurate.

Q: What is RCA in Step 8?
A: Root Cause Analysis. The AI analyzes why a test failed and categorizes it.

Q: What is a .md automation flow file?
A: A Markdown file written by AI describing test steps. Browser Bash converts these into browser actions.

Q: What is flakiness detection?
A: Identifying tests failing randomly without code changes. These need quarantining, not bug reports.

Q: What is the benefit of this pipeline?
A: Removes manual work from standard testing, speeds delivery, ensures consistency.

Q: What models can Browser Bash use?
A: Cost-effective models like DeepSeek, or any LLM via API. Goal: keep execution costs low.

Q: How does the dashboard get updated?
A: The AI agent pushes execution data (pass/fail, RCA, flakiness) into a reporting dashboard automatically.